

Báo cáo tổng hợp tiến độ

Dự án: Agentic Playwright Testing

Thời gian tổng hợp: Tuần 1 → 2026-08-31

Giai đoạn: Hoàn chỉnh pipeline 5 agent (Requirement → Test Design → Playwright Generation → Execution → Repair), nổi end-to-end và chạy LIVE trên SauceDemo; có nền đánh giá định lượng cho RQ1 và cơ chế self-healing locator có kiểm soát.

Trạng thái chung: Toàn bộ chuỗi agent hoạt động và test được offline (72 unit test xanh). Đánh giá RQ1 mới ở mức sơ bộ (chạy trên model local). Chưa triển khai persistence (SQL Server) và UI (Streamlit); đây là phần cố ý để lại cho slice sau.

1. Mục tiêu tổng thể

Xây một hệ AI tác tử (agentic) bán tự động: từ tài liệu yêu cầu sinh và kiểm chứng test E2E Playwright, với **người kiểm duyệt trong vòng lặp** (human-in-the-loop) ở mỗi cổng quyết định. Ba định hướng novelty:

- Human-governed** — agent chỉ đề xuất; con người duyệt qua các cổng AG-01...AG-05; agent không tự áp dụng thay đổi cần duyệt.
- Traceable** — truy vết Requirement → Test case → Script → Execution → Repair; đo độ phủ và phát hiện liên kết mờ côi.
- Constrained repair** — tự chữa lỗi nhưng mức rủi ro do *chính sách* quyết định (không phải LLM); không làm yếu assertion để test "xanh giả".

2. Công việc đã hoàn thành

2.1 Requirement Analysis Agent

- Trích yêu cầu có cấu trúc (Pydantic); gắn cờ mơ hồ theo rubric 7 loại kèm lý do + trích nguồn.
- Parser DOCX/PDF/TXT/MD (chèn mốc trang để truy vết).
- Chỉ phát hiện mơ hồ, **không tự làm đầy** — chờ duyệt AG-01. Fail-safe khi input rỗng.

2.2 Test Design Agent

- Từ yêu cầu **đã duyệt** sinh test case đa loại (positive/negative/boundary/error_guessing/alternative_flow).
- Gán ID truy vết REQ-...-TC-...; đo độ phủ theo loại test; cổng phê duyệt AG-02.

2.3 Playwright Generation Agent

- Chuyển test case → plan action Playwright → render script tự chứa (Jinja2).
- Chính sách locator: ưu tiên role → label → placeholder → text → test_id → css; chọn locator **khớp đúng 1**.
- "Grounding": đếm số phần tử khớp trên UI thật để phát hiện locator yếu.

2.4 Execution Agent

- Chạy script thật trong tiến trình con; thu ảnh chụp làm bằng chứng.
- Phân loại kết quả: passed / failed / error / blocked.

2.5 Repair Agent + Repair Policy

- Đề xuất bản sửa; **rủi ro do policy quyết định** (Low: locator/wait; Medium: dữ liệu/điều hướng; High: assertion/bước/kỳ vọng; Cấm: sửa nguồn website/xoá bằng chứng).
- Ngân sách sửa = 2 → vượt thì chuyển "chờ người xem xét"; mọi đề xuất đều `requires_approval`.

2.6 Self-healing locator

- Khi locator gây lúc chạy, hệ thống **dò DOM sống**, khớp mờ đúng element có ý định ban đầu, sinh locator thay thế **duy nhất** theo chính sách locator — thay cho đáp án cài sẵn.
- Đường sửa **tất định** (không cần LLM), nhưng vẫn qua repair policy + phê duyệt; fail-closed nếu không tìm được locator duy nhất.

2.7 Nền tảng dùng chung

- **Dịch vụ truy vết**: dựng chuỗi Req→TC→Script→Exec→Repair, đo requirement/executed coverage, pass rate, phát hiện orphan.
- **LLM abstraction**: một lớp chung chạy cả cloud (OpenAI/DeepSeek) và local (LM Studio/Ollama) qua `base_url`; đổi backend chỉ đổi `.env`; tự thích ứng `response_format` (`json_schema` ↔ `json_object`).
- **Bộ đánh giá định lượng**: precision/recall/F1 theo cặp (yêu cầu, loại), micro/macro, over-flag.
- **Capstone** `run_full_pipeline.py`: chạy nguyên chuỗi LIVE trên SauceDemo.

2.8 Kiểm thử & Git

- **72 unit test xanh**, chạy offline bằng LLM giả lập (deterministic), theo quy trình TDD RED→GREEN; logic lỗi (repair policy, traceability, healing) test kỹ.

3. Sản phẩm tạo ra

SẢN PHẨM	ĐƯỜNG DẪN	TRẠNG THÁI
Requirement/Ambiguity models	<code>src/models/requirement.py</code>	Hoàn thành
Test case / Approval models	<code>src/models/test_case.py</code> , <code>src/models/approval.py</code>	Hoàn thành
Playwright / Repair artifacts	<code>src/models/playwright_artifacts.py</code> , <code>src/models/repair.py</code>	Hoàn thành

SẢN PHẨM	ĐƯỜNG DẪN	TRẠNG THÁI
Requirement Analysis Agent	src/agents/requirement_agent.py	Hoàn thành
Test Design Agent	src/agents/test_design_agent.py	Hoàn thành
Playwright Generation Agent	src/agents/playwright_generation_agent.py	Hoàn thành
Execution Agent	src/agents/execution_agent.py	Hoàn thành
Repair Agent	src/agents/repair_agent.py	Hoàn thành
LLM client (cloud + local)	src/services/llm_client.py	Hoàn thành
Locator policy + Self-healing	src/services/locator_policy.py , src/services/locator_healing.py	Hoàn thành
Repair policy (constrained)	src/services/repair_policy.py	Hoàn thành
Script template + Traceability	src/services/script_template.py , src/services/traceability.py	Hoàn thành
Parser tài liệu	src/services/document_loader.py	Hoàn thành
Bộ đánh giá (metrics/dataset/evaluator)	src/evaluation/	Hoàn thành
CLI pipeline	run_requirement_agent.py , run_test_design_agent.py , run_playwright_pipeline.py , run_full_pipeline.py	Hoàn thành
Báo cáo truy vết (sinh tự động)	docs/development/traceability-report.md	Hoàn thành
Unit tests	tests/unit/	72 test xanh

4. Quyết định kỹ thuật (chọn lọc)

ID	QUYẾT ĐỊNH	LÝ DO
TD-13	LLM tách sau interface (Protocol) + MockLLMClient	Test/chạy offline; đổi provider không sửa agent
TD-14	Output agent là Pydantic có cấu trúc	Validate, lưu trữ, tái lập được
TD-16	Agent chỉ đề xuất, con người duyệt (AG-01...05)	Human-governed; kiểm soát rủi ro AI

ID	QUYẾT ĐỊNH	LÝ DO
TD-19	Dùng chung SDK cho cloud và local qua <code>base_url</code>	Không thêm dependency; đổi backend chỉ đổi <code>.env</code>
TD-24	Rủi ro repair do policy quyết định, không phải LLM	Ràng buộc khách quan; cấm làm yếu assertion
TD-25	Ngân sách sửa = 2 → <code>blocked_for_review</code>	Tránh vòng lặp sửa vô hạn
TD-26	Self-healing locator tất định từ DOM sống, fail-closed	Tự chữa từ dữ liệu trang thật; không đề xuất bừa
TD-27	Chọn locator ưu tiên role→...→css và khớp đúng 1	Locator bền theo best practice Playwright

5. Kết quả đánh giá sơ bộ (RQ1)

Kết quả chạy trên **model local (Gemma-4-12B)** nên là số liệu **sơ bộ**, dùng làm baseline để đối chiếu cloud sau. Ground truth bằng **chèn khuyết tật có chủ đích** (15 yêu cầu, đủ 6 loại + yêu cầu sạch). Đơn vị so khớp: cặp (yêu cầu, loại mơ hồ).

Ablation trên Gemma-4-12B — một chỉ dẫn prompt nâng micro-F1 ~10 lần:

CẤU HÌNH	MICRO-F1	MACRO-F1	RECALL
Trước (chưa nắn prompt)	0.07	0.17	0.08
Sau (gắn ambiguity theo từng yêu cầu)	0.73	0.78	1.00

So sánh local vs cloud :

MODEL	MICRO-P	MICRO-R	MICRO-F1	OVER-FLAG*
google/gemma-4-12b (local)	0.57	1.00	0.73	0.40
deepseek-chat (cloud)	0.34	1.00	0.51	0.80

Over-flag = tỉ lệ yêu cầu sạch bị gắn cờ nhầm (thấp là tốt).*

Nhận xét: - Bộ đánh giá **định lượng hoá** được một lỗi tinh vi (model đặt ambiguity ở mức toàn cục thay vì gắn theo từng yêu cầu); sửa xong recall đạt 1.00. - Trên dataset này, model local vừa F1 cao hơn vừa ít over-flag hơn cloud — một kết quả **cần đối chiếu thêm**, chưa kết luận.

6. Kết quả demo LIVE (capstone trên SauceDemo)

- TC-01 (đăng nhập hợp lệ):** grounding 3/3 khớp đúng 1 → thực thi **PASSED**.
- TC-02 (locator sai chủ đích):** grounding 2/3 → thực thi **FAILED** → Repair **tự chữa từ DOM sống**: `placeholder:Username` → `role:textbox[Username]`, rủi ro **LOW**, `requires_approval=true`, **agent KHÔNG tự áp dụng**.

- **Truy vết:** requirement_coverage 100% · executed_coverage 100% · pass_rate 50% (1 pass / 1 fail có chủ đích).

7. Vấn đề gặp phải / chưa giải quyết

ID	VẤN ĐỀ	MỨC ẢNH HƯỞNG	HƯỚNG XỬ LÝ
OI-13	Chưa nối database, UI, persistence	Trung bình	hiện chạy JSON + CLI
OI-16	Precision thấp ở missing_precondition / missing_expected_outcome	Trung bình	Tinh chỉnh prompt; dùng human-in-the-loop AG-01 lọc false positive
OI-17	Nhãn gold còn chủ quan	Trung bình	Nhóm review và mở rộng nhãn đa chiều
OI-18	Đánh giá mới trên một model local, chưa đối chiếu cloud	Cao	Chạy --profile cloud (DeepSeek/OpenAI) để so sánh

8. Công việc chưa hoàn thành

- Chạy và đối chiếu RQ1 trên LLM cloud
- SQL Server persistence và Streamlit + luồng duyệt AG-01...05 trên web.
- Multi-step grounding và sinh test boundary/negative sâu hơn.
- Mở rộng và review dataset gán nhãn (chiều precondition/outcome).

9. Kế hoạch tiếp theo

1. Đối chiếu RQ1 trên cloud để kết luận vững hơn.
2. Nhóm review PR #2 rồi gộp pipeline vào `develop`.
3. Đào sâu: multi-step grounding, boundary/negative generation, siết prompt + đo lại eval.
4. Bổ sung hạ tầng: Streamlit UI (review flow) + SQL Server persistence.

10. Tự đánh giá

- **Tiến độ:** Hoàn chỉnh toàn bộ chuỗi agent và chứng minh LIVE end-to-end, kể cả tình huống lỗi + tự chữa có kiểm soát — phần thể hiện rõ nhất novelty của đề tài. Persistence/UI còn để lại theo scope.
- **Chất lượng:** Kiến trúc tách lớp, output có cấu trúc, fail-closed; 72 unit test theo TDD; logic lỗi test kỹ. Cần đối chiếu cloud và review ground truth để kết luận đánh giá vững hơn.